머신러닝

UI 미니프로젝트 소개

1. 개요

배경

- 좋은 결과물을 만들기 위해 **생성형 AI를 적극 활용**하여 UI를 구현할 필요가 있다.
- AI 도구의 발전으로 **프론트엔드와 백엔드 구현 장벽이 낮아졌으며**, 이제는 두 영역을 **통합적으로 이해하고 개발**하는 역량이 요구된다.

목표

- **UI 프로세스 설계**부터 **백엔드 연동**까지 한 흐름으로 구현할 수 있는 역량을 함양한다.
- 생성형 AI를 활용해 개발 효율을 높이기 위해 **프롬프트 설계 방법**을 익힌다.

2. 조별 프로젝트 주제

1조) Object detection을 위한 데이터 라벨링 및 증강 구현

2조) 새로운 기종 및 검사 이미지 추가 로직 구현

3조) 병렬 검사를 위한 서버 구현

4조) 모델 학습 시 Active Learning 적용 구현

5조) 카메라 기반 실시간 테스트 모드 구현

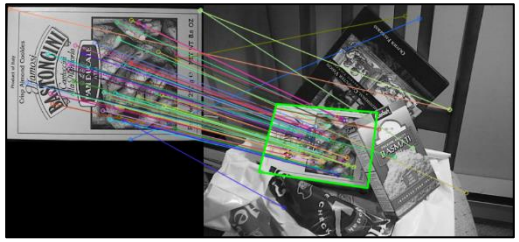
6조) 처리 로그 및 데이터 확보 구현

- **발표 시간: 조별 10분 엄수** (시간 초과 시 진행자가 종료할 수 있음)
- **발표 내용:** 프로젝트 주제를 **간단히 설명**하고, 이를 **백엔드·프론트엔드로 구현 가능한 수준**으로 제시
- **필수 : UI 흐름** + **백엔드 처리 흐름이 잘 보이도록 발표 자료를 구성**
- **시연:** UI의 **실행 화면을 짧은 영상으로 처리 프로세스가 잘 보이도록** 제시하여 설명
- **프로젝트 내용 문의 : SSK135@hanyang.ac.kr**

2. 조별 프로젝트 주제

1. Provided 알고리즘 소개

- 제공된 알고리즘은 **ORB Homography**와 **Template Matching**으로 구성
- 입력 이미지와 검사 이미지 사이 **유사도 측정**



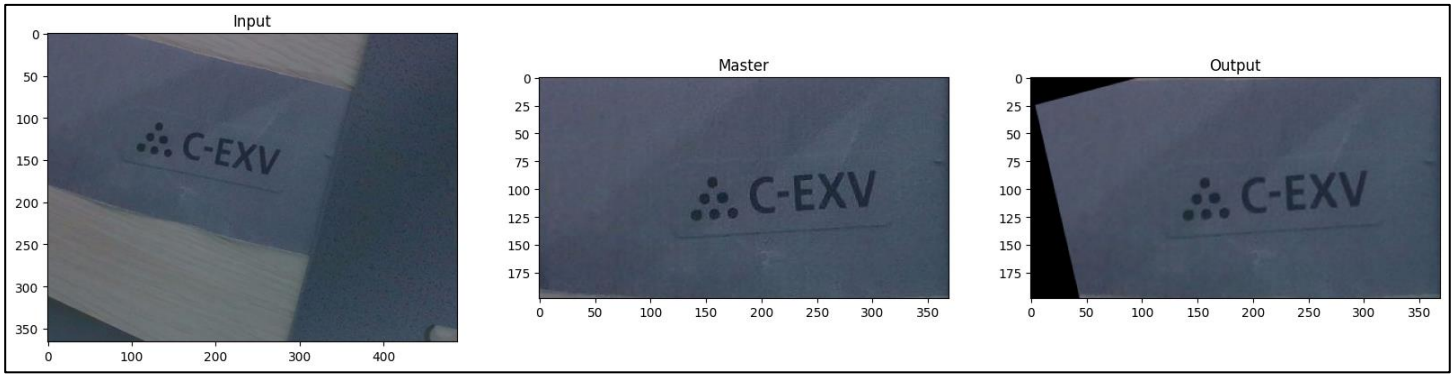
2. ORB Homography

- **특징점**을 얻어 입력 이미지와 검사 이미지 사이 특징점 매칭 수행
- **위치, 회전, 원근 차이** 보정

3. Template Matching

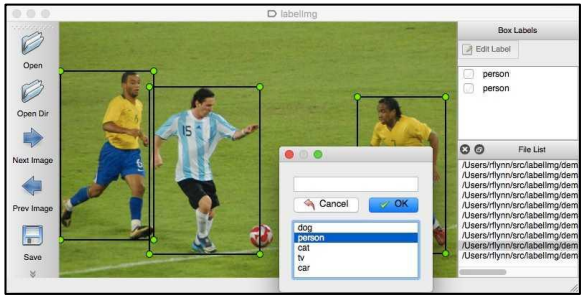
- 정렬된 이미지에서 검사 이미지와 가장 비슷한 부분 추론
- 검사 이미지의 위치를 추정 및 매칭 점수를 통해 **유사도 측정**

→ 필요 : 검사 이미지, 입력 이미지



입력 이미지 **검사 이미지** **결과 이미지**

2. 조별 프로젝트 주제



1조) Object detection을 위한 데이터 라벨링 및 증강 구현

수행 내용

- **데이터 직접 취득**
  - 라벨링할 이미지를 직접 수집하거나 촬영하여 데이터셋 구성
- **검사 이미지 및 ROI 영역 UI 구현**
  - 사용자가 시각적으로 라벨링할 수 있도록 구현
- **되돌리기(Undo) 기능 구현**
  - 잘못 지정한 라벨 또는 ROI를 이전 단계로 복구할 수 있도록 구현
- **라벨링 결과 JSON 저장**
  - 라벨 정보와 좌표값을 JSON 형식으로 저장 이후 학습 데이터로 활용 가능하도록 구조화
- **데이터 증강 옵션 구현**
  - 회전, 밝기 변화, 노이즈 추가 등 증강 기능 제공
  - 사용자가 옵션을 선택하여 다양한 데이터를 생성할 수 있도록 구성
- **저장 기능을 위한 백엔드 고려**
  - 라벨링 결과 및 증강 데이터를 저장·관리할 수 있는 백엔드 구조 설계

2. 조별 프로젝트 주제

2조) 새로운 기종 및 검사 이미지 추가 로직 구현

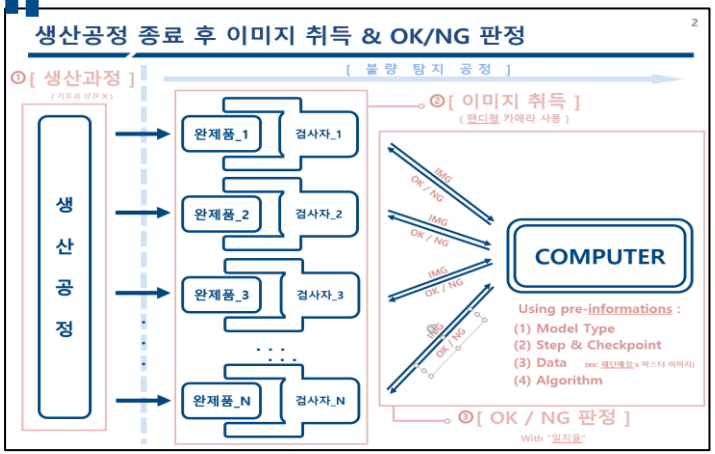
수행 내용

- **기종별 검사 구조 차이 반영**
  - 각 기종마다 검사 스텝의 순서와 검사 이미지 구성이 다를 수 있으므로, 이를 반영할 수 있는 구조 설계
- **Barcode Reading 선행 구현**
  - 어떤 기종인지 먼저 식별해야 하므로, 바코드 리딩을 첫 번째 스텝으로 구현
- **UI 기반 설정 기능 구현**
  - 사용자가 UI를 통해 기종별 스텝을 추가·수정·관리할 수 있도록 구현
  - 각 스텝에 필요한 검사 이미지를 등록할 수 있는 기능 포함
- **한 스텝 내 다중 검사 이미지 지원**
  - 하나의 스텝에 대해 여러 장의 검사 이미지가 연결될 수 있도록 설계

Step(input) 이미지

검사 이미지

2. 조별 프로젝트 주제

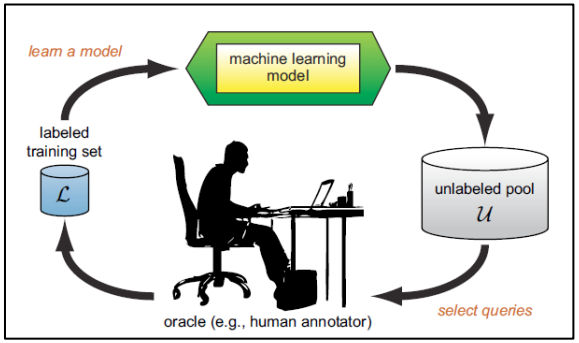


3조) 병렬 검사를 위한 서버 구현

수행 내용

- **제공된 알고리즘 기반 검사 수행**
  - 제공된 알고리즘을 활용하여 검사 기능 수행
  - 서버 요청에 따라 동일한 검사 로직이 실행될 수 있도록 구성
- **다중 PC 환경 구성**
  - 여러 PC를 네트워크로 연결하여 검사 시스템 구성
  - 각 PC가 하나의 서버 컴퓨터에 검사 요청을 보내는 구조로 설계
  - 들어온 검사 요청을 순차적으로 처리할 수 있도록 서버 로직 구현
- **모니터링 UI 설계**
  - 각 PC의 검사 결과가 메인 PC에서 한 번에 확인될 수 있도록 UI 설계
  - 각 PC에서도 자신의 검사 결과를 확인할 수 있도록 구성
  - 각 PC에서 수행된 처리 로그가 메인 PC 화면에 통합적으로 표시되도록 하여 전체 작업 흐름을 모니터링할 수 있도록 구현

2. 조별 프로젝트 주제

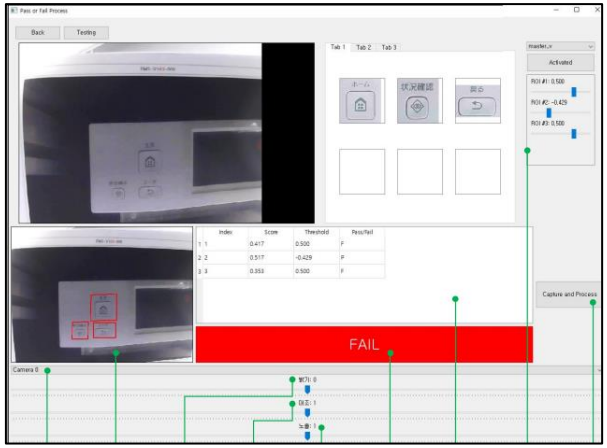


4조) 모델 학습 시 Active Learning 적용 구현

수행 내용

- **기본 학습 환경 구성**
  - MNIST 데이터셋을 이용하여 SVM 기반 분류 모델 학습 환경 구성
  - 초기 소량의 라벨링 데이터만 사용하여 1차 학습 수행
- **Active Learning 학습 구조 구현**
  - 모델이 예측에 확신이 낮은 데이터를 선택하도록 구현
  - 선택된 데이터를 사용자가 확인하고 라벨링할 수 있도록 구성
  - 새롭게 라벨링된 데이터를 학습 데이터에 추가하여 모델 재학습 수행
- **UI 기반 데이터 선택 및 학습 연결**
  - Active Learning 과정이 UI와 연결되도록 구현
  - 사용자가 UI를 통해 모델이 선택한 데이터를 확인하고 라벨링 가능하도록 구성
  - 학습 과정이 UI에서 확인될 수 있도록 구성
- **다중 모델 생성 및 관리 기능**
  - 여러 개의 모델을 생성하고 관리할 수 있는 구조 설계
  - 모델별 학습 상태 및 성능을 비교할 수 있도록 구현
  - 새로운 데이터가 추가될 때 모델을 선택하여 재학습 가능하도록 구성

2. 조별 프로젝트 주제

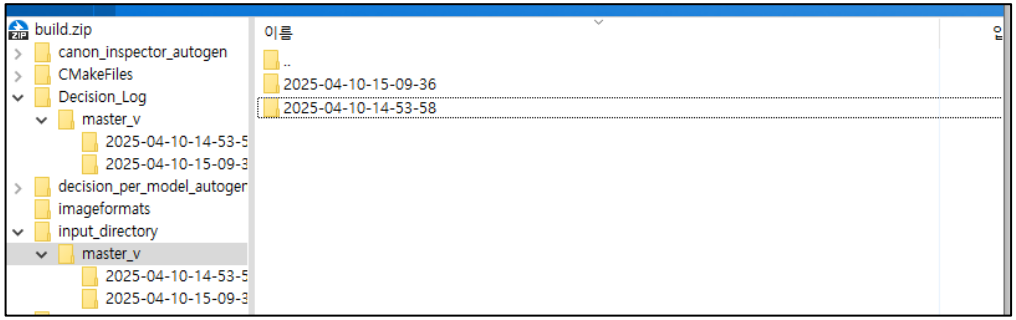


5조) 카메라 기반 실시간 테스트 모드 구현

수행 내용

- **카메라 입력 연동**
  - 제공된 카메라를 이용하여 실시간 영상 입력 환경 구성
  - 직접 데이터를 취득하며 테스트 가능한 구조 구현
- **밝기 및 대조 조정 기능 구현**
  - 입력되는 이미지에 대해 밝기와 대조를 조절할 수 있는 기능 구현
  - 사용자가 직접 값을 변경하며 결과를 바로 확인할 수 있도록 구성
- **설정 값 저장 및 불러오기**
  - 사용자가 조정한 밝기·대조 설정 값을 저장할 수 있도록 구현
  - 저장된 값을 다시 불러와 동일한 환경으로 연결할 수 있도록 구성
- **입력 이미지 위 결과 표시**
  - 제공된 알고리즘을 이용하여 검사 또는 처리 과정 수행
  - 각 검사 이미지에 대한 **Score**를 계산하고 화면에 표시
  - Score를 기준으로 **Pass / NG** 여부를 판단하여 함께 출력
  - 검사 대상 이미지가 입력 이미지 위에 직접 보이도록 구성

2. 조별 프로젝트 주제



6조) 처리 로그 및 데이터 확보 구현

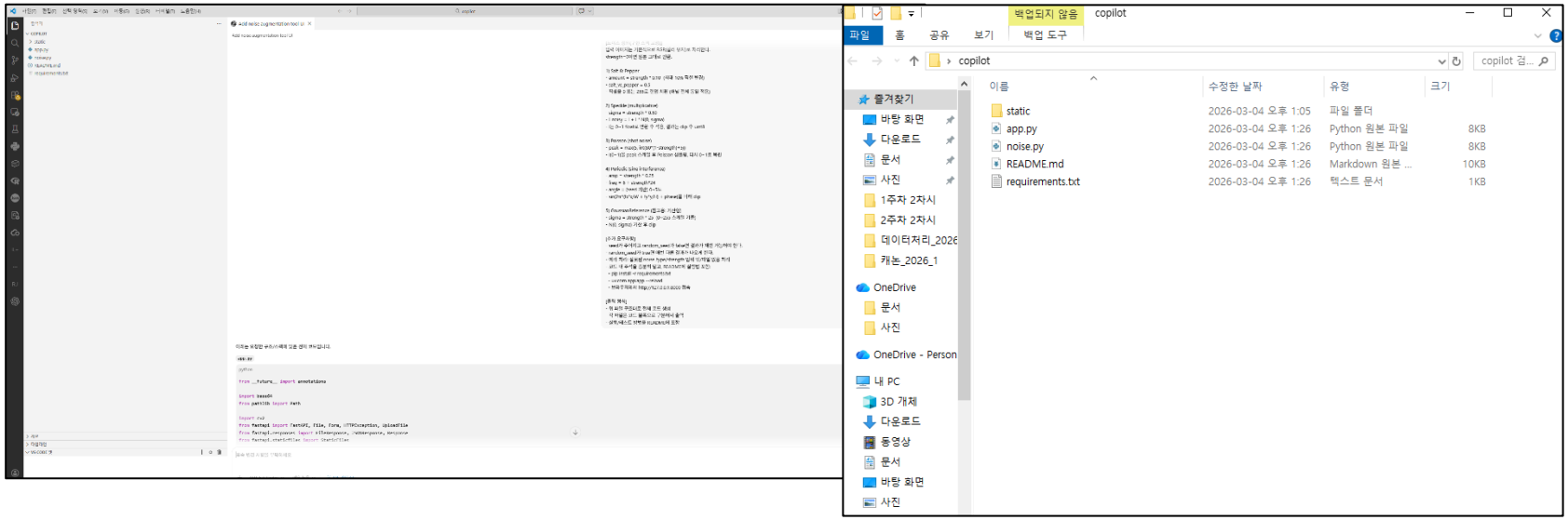
수행 내용

- **처리 로그 기록 기능 구현**
  - 검사 시각, 입력 데이터 정보, 처리 결과, 오류 발생 여부 등을 로그로 저장
  - 각 작업의 진행 과정과 결과를 추적할 수 있도록 구성
  - 제공된 알고리즘과 데이터를 이용하되, 카메라 대신 데이터 불러오기를 통해 검사 실행
  - 복수의 검사 이미지에 대해 로그가 남아야 함.
- **원본 데이터 확보를 통한 데이터베이스 구축**
  - 직접 취득한 **원본 데이터를 저장 및 관리**
  - 원본 데이터만으로도 추후 활용 가능한 **데이터베이스를 구축**할 수 있도록 설계
- **로그 및 데이터 조회 기능 구현**
  - 저장된 로그와 데이터를 다시 확인할 수 있는 구조 설계
  - 특정 작업의 처리 이력과 결과를 쉽게 조회할 수 있도록 구성
- **예외 상황 기록 및 관리**
  - 오류 발생 시 에러 내용과 발생 시점을 기록할 수 있도록 구현
  - 문제 발생 시 원인 분석과 재현이 가능하도록 로그 체계 구성

3. AI agent

프로젝트 해결을 위해 다양한 AI agent 들을 활용할 수 있음.

1. Copilot

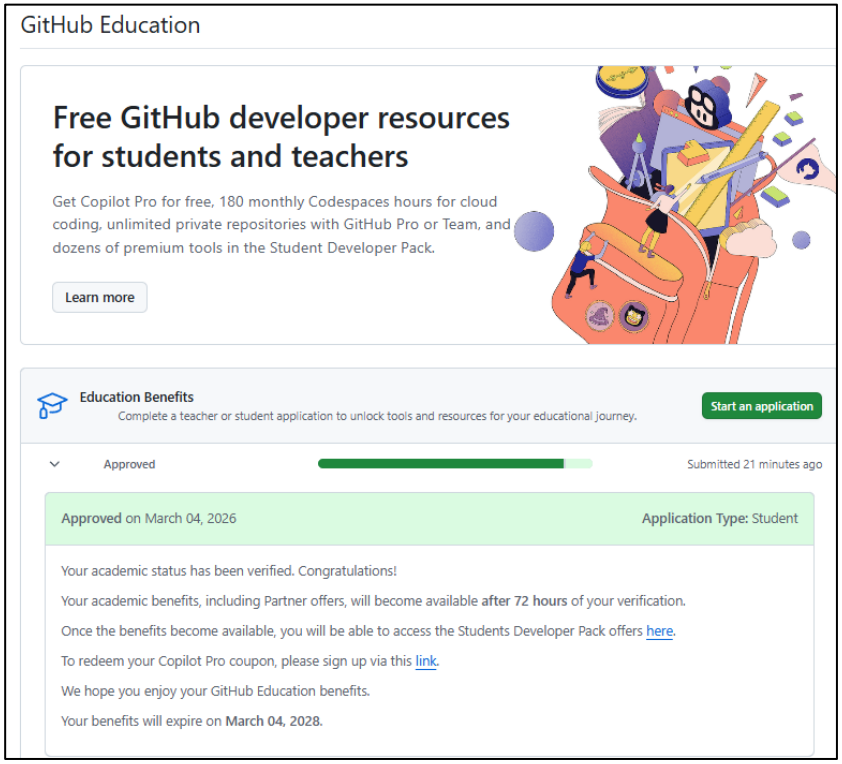


3. AI agent

프로젝트 해결을 위해 다양한 AI agent 들을 활용할 수 있음.

1. Copilot

현재 코파일럿의 경우 pro를 학생들에게 2년간 무료로 제공하고 있음.

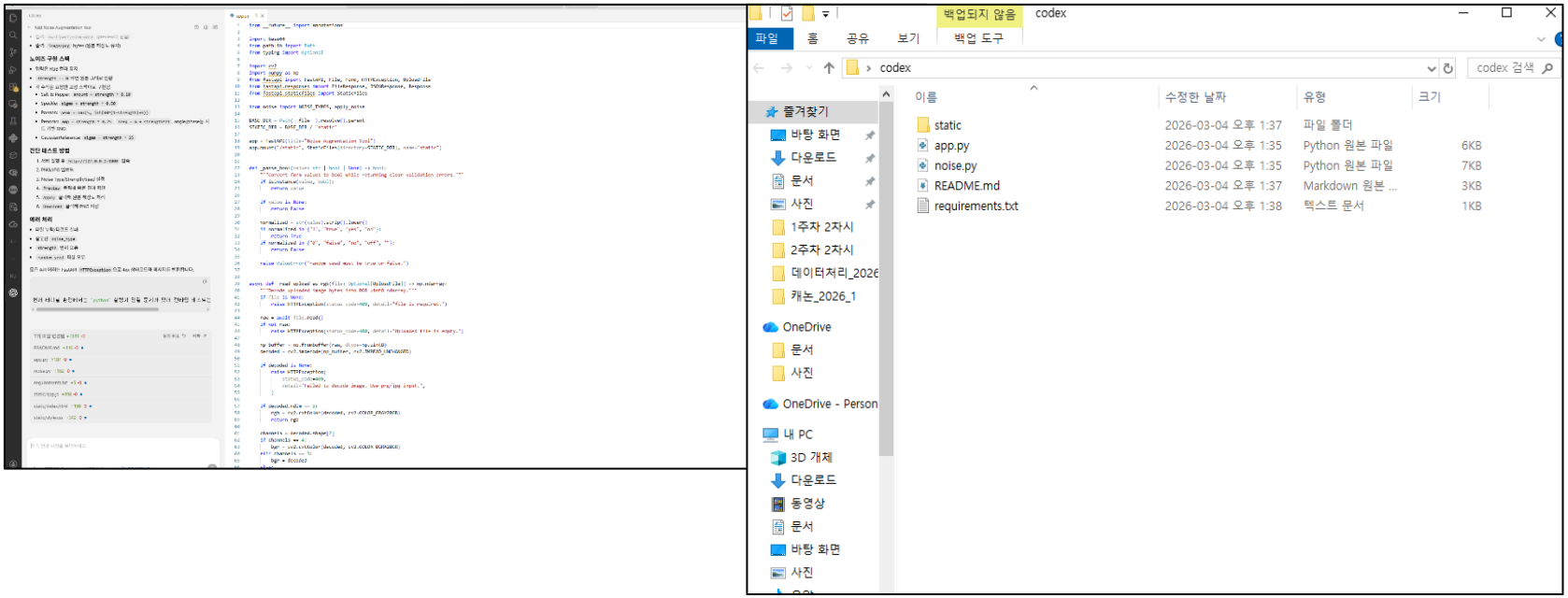


3. AI agent

프로젝트 해결을 위해 다양한 AI agent 들을 활용할 수 있음.

2. Codex

GPT를 결제하여 사용하고 있는 사람들에게 추천

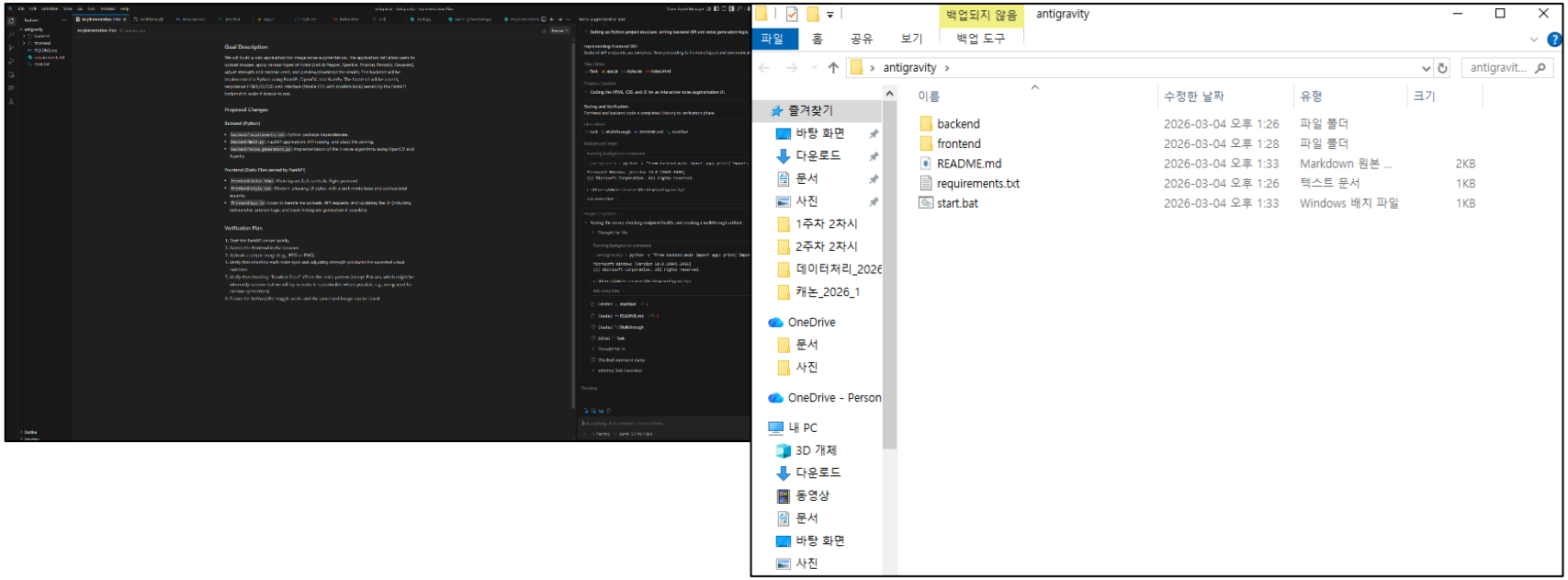


3. AI agent

프로젝트 해결을 위해 다양한 AI agent 들을 활용할 수 있음.

3. Antigravity

Gemini를 결제하고 사용하고 있는 사람들에게 추천.



3. AI agent

LLM을 이용하여 입력 프롬프트를 만들어달라고 요청

코덱스를 이용할건데 UI를 만들어서 이미지에 노이즈를 더하는 프롬프트를 짜줘

아래는 **Codex에게 그대로 붙여넣으면** "웹 UI + 백엔드(파이썬) + 이미지 노이즈 추가"까지 한 번에 생성하도록 설계한 **프롬프트 템플릿**이야. (요구사항/파일구조/노이즈 스펙/실행방법까지 강제)

Plain text

```
너는 OpenAI Codex다.
목표: 사용자가 웹 UI에서 이미지를 업로드하고, 노이즈 종류/강도/시드를 선택하면 서버에서 노이즈를 적용해 미리보기

[기술 스택 고정]
- Backend: Python FastAPI
- Frontend: 단일 HTML (Vanilla JS + CSS) 1페이지
- 이미지 처리: OpenCV(cv2) + NumPy
- 데이터 전송: multipart/form-data (이미지 업로드), JSON(옵션)
- 결과 반환: PNG bytes (Response) + 미리보기는 base64 dataURL
- 실행 환경: 로컬 실행(Windows/macOS/Linux) 가능

[파일 구조]
- app.py                  (FastAPI 서버)
- noise.py                (노이즈 함수 모음)
- static/index.html       (UI)
- static/style.css        (UI 스타일)
- static/app.js           (UI 로직)
- requirements.txt
- README.md               (실행 방법)

[UI 요구사항]
1) 제목: "Noise Augmentation Tool"
2) 좌측 컨트롤 패널
   - Image Upload (png/jpg)
   - Noise Type 드롭다운:
     - SaltPepper, Speckle, Poisson, Periodic, GaussianReference
   - Strength 슬라이더: 0.0 ~ 1.0 (step 0.01) + 숫자 표시
   - Seed 입력(정수) + "Random Seed" 체크박스
```

HANYANG
UNIVERSITY

한양대학교 ERICA 캠퍼스